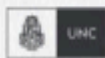


CERTIFICACIÓN UNIVERSITARIA



FCEFYN



Manual para el
alumno

Introducción a
seguridad



WE WANT
SKILLS!

mundosE

Manual para el alumno: Introducción a seguridad.....	2
Objetivo del encuentro:.....	2
1. Marco conceptual de seguridad (tríada CIA).....	3
2. Amenazas y vulnerabilidades comunes.....	4
3. Evaluación de riesgos y SDLC seguro.....	6
4. OWASP ZAP (DAST) — enfoque teórico.....	8
Bibliografía.....	9

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontrarás un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Introducción a seguridad

Objetivo del encuentro:

Este manual guía por los cuatro ejes de la clase: **Marco conceptual de seguridad (tríada CIA), Amenazas y vulnerabilidades comunes, Evaluación de riesgos y SDLC seguro, y OWASP ZAP (enfoque teórico, DAST)**. El objetivo es que construyas un vocabulario común, aprendas a reconocer señales de riesgo, practiques cómo priorizarlas y sepas interpretar un reporte de DAST como ZAP dentro de un proceso de desarrollo seguro.

Contenido a desarrollar:

- ✓ Entender la **tríada CIA** (Confidencialidad, Integridad, Disponibilidad) y cómo se traduce en decisiones técnicas.
- ✓ Identificar **amenazas** típicas y mapearlas a **vulnerabilidades** frecuentes en aplicaciones web.
- ✓ Aplicar una **matriz de probabilidad × impacto** para **priorizar riesgos** y vincularla al **SDLC seguro**.
- ✓ Comprender, **en teoría**, qué hace **OWASP ZAP** (DAST), qué detecta y cuáles son sus límites.
- ✓ Conectar hallazgos con **mitigaciones primarias** (p. ej., parametrización, escape de salida, cabeceras de seguridad, CSRF tokens).
- ✓ Desarrollar criterio para **leer reportes**, separar ruido de señal y **planificar mejoras** incrementales.

1. Marco conceptual de seguridad (tríada CIA)

La **seguridad en el desarrollo de software** es la capacidad de un sistema para proteger adecuadamente los datos y los servicios que administra, de modo que solo puedan acceder y modificarlos quienes están autorizados, y que permanezcan disponibles cuando se los necesita.

Esta idea se plasma en la tríada CIA: Confidencialidad, Integridad y Disponibilidad.

La **Confidencialidad** busca evitar que información sensible (por ejemplo, datos personales o financieros) se divulgue a actores no autorizados. Esto abarca tanto miradas indebidas como fugas masivas. En la práctica, se materializa con controles como autenticación robusta (idealmente con múltiples factores), autorización en servidor (roles/atributos), cifrado en tránsito (TLS) y, cuando corresponde, cifrado en reposo. Si bien “cifrar todo” suena atractivo, no reemplaza a los demás controles: todavía necesitas definir quién puede ver qué y auditar el uso.

La **Integridad** protege contra modificaciones o destrucciones no autorizadas de datos o funciones. Una transferencia que altera su monto sin permiso, una cuenta cuyo estado se cambia de manera ilegítima, un archivo corrompido: todos son ejemplos de fallas de integridad. Este principio se apoya en la validación estricta de entradas (listas blancas), consultas parametrizadas o uso de ORM (para no concatenar SQL), firmas digitales/códigos de verificación para detectar alteraciones y en prácticas de colaboración como las revisiones de código con foco en cambios sensibles. También importa la integridad operacional: que los procedimientos (por ejemplo, migraciones o scripts de mantenimiento) no introduzcan inconsistencias por error humano.

La **Disponibilidad** procura que el sistema funcione y responda con confiabilidad dentro de los niveles acordados. No se limita a “que no se caiga”: también abarca que no se degrade hasta volverse impráctico. Entre las amenazas están los DDoS, picos de tráfico legítimos, *bugs* que consumen recursos, o configuraciones deficientes (*timeouts* eternos, bloqueos, *deadlocks*). Las defensas combinan capacidad (escalado, colas, cachés), resiliencia (*circuit breakers*, *timeouts* razonables), limpieza (*rate limiting*) y plan de continuidad (*backups* probados, *runbooks* de recuperación). La disponibilidad se vincula con experiencia de usuario: si el servicio no está cuando se lo necesita, se pierde confianza.

Más allá de la tríada, hay conceptos complementarios que enriquecen el marco:

- **autenticación** (verificar identidades), autorización (definir y hacer cumplir permisos);
- **trazabilidad** (registros útiles, sin exponer datos sensibles, para investigar);

- **no repudio** (evidencias que impiden negar acciones realizadas); y,
- **privacidad** (derechos y obligaciones respecto de datos personales a lo largo de su ciclo).

Un error frecuente es creer que “siempre que haya cifrado, la seguridad está resuelta”: el cifrado protege la confidencialidad en tránsito/en reposo, pero no asegura por sí mismo integridad, disponibilidad ni autorización correcta.

¿Cómo se baja esto al trabajo cotidiano?

Primero, entendiendo que la seguridad no se agrega al final. Si un equipo intenta “parchar” al cierre, suele enfrentar re-trabajos costosos y riesgos residuales. La práctica moderna promueve **security by design** (decisiones conscientes en requisitos/diseño), **security by default** (configuraciones seguras por defecto) y **shift-left** (detectar antes, cuando más barato es corregir).

Segundo, adoptando patrones de codificación segura:

- tratar toda entrada como no confiable hasta validarla;
- codificar la salida según su contexto (HTML, atributo, URL, JavaScript) para prevenir XSS;
- usar consultas parametrizadas u ORM para evitar SQLi;
- manejar errores sin filtrar información sensible; y,
- gestionar secretos por fuera del repo (vaults/identidades de workload).

Tercero, verificando con disciplina: revisiones de código con *checklist* de seguridad; SAST (análisis estático) para detectar patrones peligrosos temprano; DAST (escaneo dinámico) para observar la app en ejecución; SCA/SBOM para entender qué dependencias reales se usan y qué riesgos traen.

El equilibrio CIA también se negocia con experiencia de usuario y negocio. Aumentar la fricción (por ejemplo, MFA obligatorio) puede mejorar confidencialidad, pero afectar adopción si no se comunica y diseña bien; deshabilitar por completo funciones puede “romper” la disponibilidad. El objetivo no es “maximizar” cada vértice de CIA sin mirar el contexto, sino definir niveles aceptables de riesgo, documentar decisiones y revisarlas periódicamente.

Finalmente, **la seguridad es una práctica continua**: cambian las amenazas, aparecen nuevas dependencias, evolucionan las regulaciones. Por eso, además de construir bien, se debe medir (métricas simples: hallazgos críticos abiertos, tiempo de remediación), aprender (*post-mortems* sin culpa) y mejorar. Este enfoque se integra de manera natural en equipos que trabajan con iteraciones cortas y entrega continua.

Material para profundizar

- OWASP Top 10: <https://owasp.org/www-project-top-ten/>

- OWASP Cheat Sheet Series: <https://cheatsheetseries.owasp.org/>
- ISO/IEC 27001: <https://www.iso.org/standard/27001>

2. Amenazas y vulnerabilidades comunes

Diferenciar amenazas de vulnerabilidades ayuda a pensar bien las defensas. Las **amenazas** describen actores y tácticas que intentan causar daño (lucrar, espiar, vandalizar o explotar errores humanos). Las **vulnerabilidades** son debilidades explotables en diseño, implementación o configuración que permiten a esas amenazas concretarse. Aunque las amenazas muten de disfraz, muchas vulnerabilidades se repiten por causas raíz previsibles: entradas no validadas, salidas sin escape, autorización en el cliente, secretos en el repositorio, dependencias desactualizadas o mal elegidas.

Entre las amenazas más frecuentes: *malware* (*ransomware* que cifra archivos/sistemas para extorsionar; troyanos que se camuflan; gusanos que se propagan), *phishing* y *spear-phishing* (engaño dirigido o masivo por correo/mensajería para capturar credenciales o ejecutar acciones), ingeniería social (aprovecha sesgos de urgencia/autoridad/reciprocidad), DDoS (saturación de recursos para degradar/derribar), abuso de APIs (enumeración de IDs, fuerza bruta de *tokens*, *bypass* de límites) y cadena de suministro (paquetes/pipelines comprometidos o *typosquatting*). Motivaciones disímiles convergen en afectar CIA: confidencialidad (filtraciones), integridad (alteraciones) y disponibilidad (caídas/degradaciones).

En vulnerabilidades, enfoca las más vistas en aplicaciones web. SQLi ocurre cuando se construyen consultas concatenando datos de entrada; su mitigación principal son consultas parametrizadas u ORM, validación estricta y mínimos privilegios en la base. XSS aparece cuando la salida no se codifica según el contexto (HTML, atributo, JS); se mitiga con escape contextual, plantillas seguras y CSP acotada; validar entrada ayuda, pero no reemplaza el escape de salida. CSRF fuerza a un usuario autenticado a ejecutar acciones no deseadas; se evita con tokens anti-CSRF únicos por sesión, cookies **SameSite** y verificación de origen. SSRF explota que el servidor haga *requests* a destinos elegidos por el atacante (p. ej., metadatos de nube); se mitiga con listas blancas de destinos, bloqueo de metadatos y *timeouts*. Broken Access Control se da cuando la autorización se confía al cliente o no se verifica a nivel de objeto; la mitigación es chequear en servidor por recurso/acción y probar cambio de ID en QA.

La gestión de sesiones y autenticación introduce riesgos si las *cookies* no tienen **HttpOnly/Secure/SameSite**, si las sesiones no expiran, o si las contraseñas no usan hashing adecuado (p. ej., Argon2/bcrypt/scrypt). La exposición de datos ocurre con

respuestas verbosas, serializaciones inseguras o *logs* con información sensible; la mitigación pasa por minimización de datos, máscaras y políticas de *logging*. A esto se suman dependencias de terceros: una librería vulnerable puede introducir un CVE crítico en tu *app* aunque tu código “propio” sea correcto. Generar un SBOM e implementar SCA permite saber qué paquetes/versiones reales corres y qué riesgos traen.

Para entrenar el “radar”: si un input con comillas “rompe” una pantalla, piensa en SQLi/XSS; si cambiando un parámetro en la URL ves datos de otro usuario, piensa en control de acceso; si un *request* a una URL controlada por un usuario termina accediendo a recursos internos, piensa en SSRF; si un *form* ejecuta acciones sensibles sin confirmación y sin token, piensa en CSRF. Con práctica, vas a vincular cada síntoma con causa raíz y mitigación.

No todo requiere grandes inversiones. Algunas medidas de alto retorno son: cabeceras de seguridad en el reverse proxy (CSP acotada, X-Frame-Options, X-Content-Type-Options, Referrer-Policy; HSTS si hay TLS), migrar SQL concatenado a parametrizadas/ORM, agregar tokens anti-CSRF y configurar cookies con [HttpOnly/Secure/SameSite](#). Suma gestión de secretos (no claves en repos; usar vaults o identidades de workload), hardening (desactivar features innecesarios; permisos mínimos) y observabilidad (logs útiles sin datos sensibles, métricas y alertas) para detectar anomalías.

Material para profundizar

- OWASP Top 10: <https://owasp.org/www-project-top-ten/>
- OWASP ASVS: <https://owasp.org/www-project-application-security-verification-standard/>
- OWASP Cheat Sheet Series: <https://cheatsheetseries.owasp.org/>
- NIST Cybersecurity Framework: <https://www.nist.gov/cyberframework>

3. Evaluación de riesgos y SDLC seguro

Evaluar riesgos es decidir dónde enfocarte primero. Un método mínimo, transparente y accionable es la matriz de probabilidad × impacto (3×3 o 5×5). Probabilidad se estima por exposición a Internet, complejidad del módulo, historial de fallos, volumen de uso, facilidad de explotación y señales del entorno (por ejemplo, *endpoints* de autenticación siempre son objetivo). Impacto se calibra por relación con CIA (¿se filtran datos?, ¿se alteran saldos?, ¿se cae el servicio?), cumplimiento/regulación, reputación, y esfuerzo de recuperación (¿cuánto *downtime*?, ¿cuántos usuarios afectados?). Con esa grilla ubicas riesgos y priorizas: primero, alta/alta; luego, alta/media; después, media/alta, etc. Evita debates abstractos: usa ejemplos y datos (*logs*, telemetría, incidentes pasados).

Integra la priorización con el SDLC seguro. En **Requisitos**, identifica activos (datos,

procesos, integraciones) y define requisitos no funcionales de seguridad: cifrado de canal, logging sin datos sensibles, retención/expurgo responsable, control de acceso por defecto restrictivo. En Diseño, practica modelado de amenazas (p. ej., STRIDE: suplantación, manipulación, repudio, divulgación, denegación, escalamiento) y define fronteras de confianza (qué componentes confías, qué orígenes permites).

En **Implementación**, aplica patrones de codificación segura (validación de entrada; escape de salida por contexto; ORM/parametrizadas; manejo seguro de archivos; sanitización; gestión de secretos; sesiones/*cookies* seguras) y mantén revisiones de código con checklist. En **Verificación**, combina SAST (patrones peligrosos en código), DAST (comportamientos en ejecución), SCA/SBOM (dependencias reales) y pruebas de autorización a nivel de objeto (que cambiar un ID no te permita tocar datos ajenos). En Despliegue, practica *hardening* (configuraciones seguras, desactivar por defecto), firmas/atestaciones de artefactos y políticas de admisión en infraestructura (p. ej., solo imágenes firmadas con SBOM). En Operación, instrumenta observabilidad (*logs* útiles sin datos sensibles, métricas, alertas) y ejecuta un proceso de respuesta a incidentes (detectar → contener → erradicar → recuperar → aprender).

La evaluación de riesgos no termina en un *ranking*: necesita gestión disciplinada. Cada hallazgo con dueño, severidad, fecha objetivo y acción (mitigar, corregir, aceptar, transferir). Define criterios de corte (ej.: “no se despliega si hay vulnerabilidades críticas sin mitigación/justificación formal”) y excepciones con caducidad (si aceptas un riesgo temporalmente, fija fecha límite y contramedidas). Un tablero visible (Kanban) ayuda a sostener el foco y evitar la “fatiga de reportes”.

Piensa el ROI de cada control: algunas mejoras baratas y efectivas (alto retorno) son cabeceras de seguridad en el reverse proxy (CSP acotada, X-Frame-Options, X-Content-Type-Options, Referrer-Policy; HSTS si hay TLS), migrar SQL concatenado a parametrizadas, agregar tokens anti-CSRF y configurar cookies con [HttpOnly/Secure/SameSite](#). Otras demandan más trabajo (p. ej., control de acceso a nivel de objeto), pero son críticas. Con la matriz, elegís el siguiente paso que más reduce riesgo por unidad de esfuerzo.

Para sostener la mejora, define métricas simples: hallazgos críticos abiertos, MTTR para severidades altas, reincidencia por categoría, porcentaje de PRs con *checklist* de seguridad completo. Úsalas como señales, no como metas rígidas; la métrica sirve para enfocar conversaciones y detectar cuellos de botella. Finalmente, cultiva una cultura de aprendizaje: *post-mortems* sin culpas (¿qué aprendimos?, ¿qué cambiamos?), demo de seguridad en *showcases* y difusión de patrones de incidente (qué vimos, cómo lo evitamos). Esto mantiene vivo el SDLC seguro y evita que la seguridad se convierta en una “auditoría anual” desconectada del desarrollo.

Material para profundizar

- NIST SP 800-30 (Risk Assessment):
<https://csrc.nist.gov/publications/detail/sp/800-30/rev-1/final>
- OWASP SAMM: <https://owasp samm.org/>
- OWASP ASVS:
<https://owasp.org/www-project-application-security-verification-standard/>

4. OWASP ZAP (DAST) — enfoque teórico

OWASP ZAP (Zed Attack Proxy) es una herramienta de DAST (Dynamic Application Security Testing). Analiza la aplicación en ejecución desde la perspectiva de un cliente para observar respuestas y, según el caso, inducir comportamientos que revelen vulnerabilidades. A diferencia de SAST (que inspecciona código sin ejecutarlo) y SCA/SBOM (que inventaria dependencias y licencias), ZAP se ubica “desde afuera”, como lo haría un usuario o un atacante, y su valor conceptual es evidenciar problemas en el flujo HTTP real: cabeceras de seguridad ausentes, salidas sin escape, *endpoints* expuestos, mensajes de error verbosos, fallas de autorización que se manifiestan en respuestas, etc.

ZAP opera (teóricamente) en dos modos: escaneo pasivo y escaneo activo. El pasivo (a menudo llamado “*baseline*”) inspecciona el tráfico que pasa por el proxy sin enviar *payloads* intrusivos; detecta, por ejemplo, cabeceras de seguridad faltantes o patrones sospechosos. Es rápido y de bajo riesgo, útil para chequeos frecuentes y para establecer “higiene” mínima. El activo (*full scan*) ejecuta pruebas específicas (*payloads*) para intentar confirmar vulnerabilidades como XSS, SQLi o Path Traversal. Aporta profundidad, pero debe reservarse a entornos controlados por sus posibles efectos colaterales.

Para “entender” ZAP desde la teoría conviene revisar sus componentes mentales. Primero, el proxy de interceptación: el canal por el que pasa el tráfico cliente-aplicación y que permite trazar requests/responses para análisis posterior. Segundo, los mecanismos de descubrimiento de superficie: el Spider tradicional sigue enlaces HTML; el AJAX Spider está orientado a SPAs, donde el DOM y las rutas se generan dinámicamente. Tercero, la política de escaneo, que define qué probar, con qué intensidad y qué excluir; es el contrato entre cobertura, tiempo y riesgo de impacto. Cuarto, el modelo de alertas: cada hallazgo queda con tipo (p. ej., “Reflected XSS”), riesgo (High/Medium/Low/Informational), confianza (probabilidad de que sea verdadero positivo), evidencia (fragmentos relevantes de request/response) y recomendación.

La lectura crítica de alertas es central: toma cada alerta como un objeto con campos. Síntoma (qué se observó), hipótesis de causa raíz (por qué ocurre), impacto sobre CIA, y mitigación genérica (qué práctica lo corrige). Esta lectura teórica no requiere operar la herramienta: te prepara para priorizar y hablar con el

equipo. Por ejemplo, una alerta de “Missing X-Content-Type-Options” suele tener riesgo bajo/medio y mitigación simple (agregar cabecera); un Broken Access Control con evidencia clara es de alto riesgo y probablemente demande cambios de diseño/código.

Conoce los límites y supuestos: ZAP puede no cubrir rutas si requieren flujos complejos de autenticación, MFA o estados específicos; ahí aparecen los contextos (representar dominios/roles/credenciales) para guiar la navegación. SPAs y APIs exigen estrategias de descubrimiento distintas; a veces se necesita definir *endpoints* manualmente o scripts de autenticación. ZAP tampoco ve errores de diseño que no se manifiestan en la superficie observable (por eso complementa, no reemplaza, a SAST/SCA/IAST).

La ética es parte del enfoque: un DAST puede provocar carga extra o llenar *logs*. Por lo tanto, se debe definir alcance (dominios/rutas), contar con autorización explícita, elegir ventanas adecuadas y comunicar a los equipos involucrados. En productivo, el escaneo activo puede afectar disponibilidad o datos; se reserva para ambientes de prueba. Conceptualmente, ZAP no “certifica seguridad”: produce evidencia que alimenta la gestión de riesgos (priorización, dueños, remediaciones y reverificación).

Finalmente, aprende a priorizar: pregúntate por cada alerta qué impacto podría tener sobre CIA, qué causa raíz sugiere y cuál es la primera mitigación razonable. Con esa disciplina, un reporte deja de ser una lista y se convierte en una hoja de ruta.

Material para profundizar

- OWASP ZAP — documentación: <https://www.zaproxy.org/docs/>
- OWASP Web Security Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>
- OWASP Cheat Sheet Series (XSS/SQLi/CSRF/SSRF/AuthN-AuthZ): <https://cheatsheetseries.owasp.org/>

Bibliografía

- OWASP Foundation. (2021). *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. <https://owasp.org/www-project-top-ten/>
- OWASP Foundation. (s. f.). *Zed Attack Proxy (ZAP) — Documentation*. <https://www.zaproxy.org/docs/>
- OWASP Foundation. (s. f.). *Web Security Testing Guide*. <https://owasp.org/www-project-web-security-testing-guide/>
- OWASP Foundation. (s. f.). *Application Security Verification Standard (ASVS)*. <https://owasp.org/www-project-application-security-verification-standard/>
- OWASP Foundation. (s. f.). *Cheat Sheet Series*. <https://cheatsheetseries.owasp.org/>
- National Institute of Standards and Technology. (2012). *SP 800-30 Rev.1: Guide for Conducting Risk Assessments*. <https://csrc.nist.gov/publications/detail/sp/800-30/rev-1/final>
- National Institute of Standards and Technology. (2022). *Secure Software Development Framework (SSDF), SP 800-218*.
- International Organization for Standardization. (2022). *ISO/IEC 27001 — Information security management systems*.